



Department of Computer Science and Engineering (Data Science)

Subject: Design and Analysis of Algorithms Laboratory (DJS23DPC302L)

(A.Y. 2026-2027)

Experiment 1

(MERGE SORT)

AIM: Write a program to implement merge sort and analyze its time complexity.

THEORY:

Merge sort algorithms are based on a divide and conquer strategy. First, the sequence to be sorted is decomposed into two halves (Divide). Each half is sorted independently (Conquer). Then the two sorted halves are merged to a sorted sequence (Combine). Its worst-case running time has a lower order of growth than insertion sort. Since we are dealing with subproblems, we state each subproblem as sorting a subarray $A[p \dots r]$. Initially, $p = 1$ and $r = n$, but these values change as we recurse through sub problems.

To sort $A[p \dots r]$:

1. Divide Step

If a given array A has zero or one element, simply return; it is already sorted. Otherwise, split $A[p \dots r]$ into two sub arrays $A[p \dots q]$ and $A[q + 1 \dots r]$, each containing about half of the elements of $A[p \dots r]$. That is, q is the halfway point of $A[p \dots r]$.

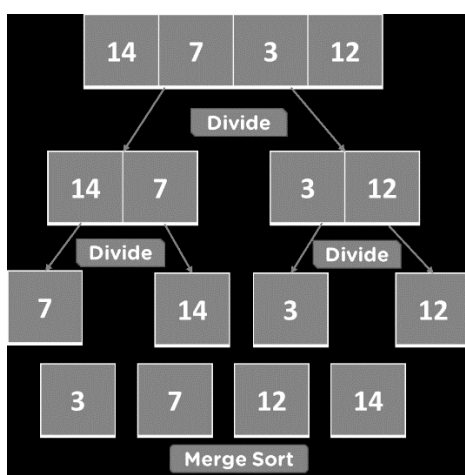
2. Conquer Step

Conquer by recursively sorting the two sub arrays $A[p \dots q]$ and $A[q + 1 \dots r]$.

3. Combine Step

Combine the elements back in $A[p \dots r]$ by merging the two sorted sub arrays $A[p \dots q]$ and $A[q + 1 \dots r]$ into a sorted sequence. To accomplish this step, we will define a procedure MERGE (A, p, q, r).

Example:





Department of Computer Science and Engineering (Data Science)

ALGORITHM:

MergeSort(int A[0....n-1,low,high) :

```
    if (low<high) then
    {
        mid ← (low+high)/2
        MergeSort (A,low,mid)
        MergeSort (A,mid+1, high)
        Combine(A,low,mid,high)
    }
```

Combine(int A[0....n-1,low,high) :

```
    k ← low
    i ← low
    j ← mid+1
    while(i<= mid && j<=high) do
    {
        if (A[i]<=A[j] then
        {
            temp[k] ← A[i]
            i←i+1
            k←k+1
        }
        else
        {
            temp[k] ← A[j]
            j←j+1
            k←k+1
        }
    }
    while (i<=mid) do
    {
        temp[k] ← A[i]
        i←i+1
        k←k+1
    }
    while (j<=high) do
    {
        temp[k] ← A[j]
        j←j+1
        k←k+1
    }
```



Shri Vile Parle Kelavani Mandal's

DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING

(Autonomous College Affiliated to the University of Mumbai)

NAAC Accredited with "A" Grade (CGPA : 3.18)



Department of Computer Science and Engineering (Data Science)

CODE:

OUTPUT:

Lab Assignment to do:

Analyze the time complexity of Merge Sort using both the Master Theorem and the Substitution Method.

CONCLUSION:

Merge Sort does not exhibit distinct best or worst cases, as it performs consistently regardless of input order. The algorithm makes $\log n$ levels of division until subarrays of size 1 are reached. At each level, merging takes $O(n)$ time, resulting in an overall time complexity of $O(n \log n)$. However, it requires auxiliary space equal to the size of the original array..